Unit - I

1. Compare and Contrast Four Fundamental Python Data Structures

Lists:

- **Definition:** Ordered, mutable collections of elements.
- **Syntax:** `my_list = [1, 2, 3, 4]`
- **Use Cases:** Storing ordered collections of items, allowing duplicates.
- **Performance:** Fast index-based access, but slow for insertions and deletions in the middle.
- **Suitability:** Best for cases where order matters and you need dynamic sizing.

Tuples:

- **Definition:** Ordered, immutable collections of elements.
- **Syntax:** `my_tuple = (1, 2, 3, 4)`
- **Use Cases:** Storing fixed collections of items, ensuring data integrity.
- **Performance:** Faster than lists due to immutability; less memory usage.
- **Suitability:** Best for fixed collections of items where immutability is required.

Dictionaries:

- **Definition:** Unordered collections of key-value pairs.
- **Syntax:** `my_dict = {'a': 1, 'b': 2, 'c': 3}`
- **Use Cases:** Mapping unique keys to values, fast lookups.
- **Performance:** Fast average-time complexity for lookups, insertions, and deletions.
- **Suitability:** Best for associative arrays, where fast lookups by key are necessary.

Sets:

- **Definition:** Unordered collections of unique elements.
- **Syntax:** `my_set = {1, 2, 3, 4}`
- **Use Cases:** Removing duplicates from a collection, set operations (union, intersection).
- **Performance:** Fast membership testing, insertion, and deletion.
- **Suitability:** Best for membership testing and ensuring unique elements.

2. Python Interaction with the Operating System for File Operations

Python provides several built-in functions and libraries to interact with the operating system, particularly for file operations.

Reading from and Writing to Files:

Reading:

```
with open('example.txt', 'r') as file:
    content = file.read()
print(content)
```

-

Writing:

```
with open('example.txt', 'w') as file:
    file.write('Hello, World!')
```

-

Manipulating File Paths and Directories:

os Library:

```
import os

# Creating a directory
os.makedirs('my_directory', exist_ok=True)

# Listing files in a directory
files = os.listdir('my_directory')
print(files)

# Getting the current working directory
cwd = os.getcwd()
print(cwd)
```

-

Handling Exceptions:

Example:

```
try:
    with open('non_existent_file.txt', 'r') as file:
        content = file.read()
except FileNotFoundError as e:
    print(f"Error: {e}")
```

-

3. IPython Shell and Its Advantages Over the Standard Python Shell

IPython Shell:

- **Definition:** An enhanced interactive Python shell.
- **Features:**
  - **Tab Completion:** For functions and variables.
  - **Object Introspection:** Using `?` and `??` to get information about objects.
  - **Magic Commands:** Special commands starting with `%` for additional functionality (`%timeit`, `%lsmagic`).
  - **Better Tracebacks:** Enhanced error messages and tracebacks.
  - **Integration with Jupyter Notebooks:** Rich media, inline plotting.

Advantages for Data Analysis:

- **Interactive Computing:** Quickly test and visualize data.
- **Rich Media Output:** Display plots, images, videos inline.
- **Magic Commands:** Simplify tasks like timing code execution (`%timeit`), debugging (`%debug`), and running system commands.

4. Control Flow Conditions in Python

Control flow statements manage the execution order of statements in Python.

If Statements:

```
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
```

For Loops:

```
for i in range(5):
    print(i)
```

While Loops:

```
count = 0
```

```
while count < 5:
    print(count)
    count += 1
```

Break and Continue:

```
for i in range(10):
    if i == 5:
        break
    if i % 2 == 0:
        continue
    print(i)
```

Try-Except Blocks:

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
finally:
    print("This will always execute")
```

5. Explain About Built-in Sequence Functions in Python

Python provides several built-in functions to operate on sequences like lists, tuples, and strings.

`len()`: Returns the number of items in a sequence.

```
my_list = [1, 2, 3, 4]
print(len(my_list))  # Output: 4
```

-

`max()` and **`min()`**: Return the maximum and minimum values in a sequence.

```
print(max(my_list))  # Output: 4
print(min(my_list))  # Output: 1
```

-

`sum()`: Returns the sum of all items in a sequence.

```
print(sum(my_list))  # Output: 10
```

-

`sorted()`: Returns a new sorted list from the elements of any sequence.

```
print(sorted(my_list))  # Output: [1, 2, 3, 4]
```

-

`reversed()`: Returns an iterator that accesses the given sequence in the reverse order.

```
print(list(reversed(my_list)))  # Output: [4, 3, 2, 1]
```

-

`enumerate()`: Adds a counter to an iterable and returns it as an enumerate object.

```
for index, value in enumerate(my_list):
    print(index, value)
# Output:
# 0 1
# 1 2
# 2 3
# 3 4
```

-

6. Describe the Language Semantics of the Python Language

Language semantics refer to the meaning and interpretation of the elements and structures in a language. In Python, the semantics define how Python code behaves and what the constructs in the language mean.

Indentation: Python uses indentation to define the scope of loops, conditionals, functions, and classes.

```
if True:
    print("This is indented")
```

-

Dynamic Typing: Variable types are determined at runtime and can change.

```
a = 10  # a is an integer
a = "Hello"  # Now a is a string
```

-

Mutable and Immutable Types: Some types (like lists and dictionaries) are mutable, while others (like strings and tuples) are immutable.

```
my_list = [1, 2, 3]
my_list[0] = 10  # This is allowed (mutable)

my_tuple = (1, 2, 3)
my_tuple[0] = 10  # This will raise an error (immutable)
```

-

Functions as First-Class Objects: Functions in Python can be passed as arguments, returned from other functions, and assigned to variables.

```
def my_function():
    return "Hello"

func = my_function
print(func())  # Output: Hello
```

-

Exceptions: Python uses exceptions to handle errors.

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

-

7. What Are the Built-in Data Structures in Python?

Python has several built-in data structures that make it easy to manage data.

Lists: Ordered, mutable collections of elements.

```
my_list = [1, 2, 3]
```

-

Tuples: Ordered, immutable collections of elements.

```
my_tuple = (1, 2, 3)
```

-

Dictionaries: Unordered collections of key-value pairs.
```
my_dict = {'a': 1, 'b': 2}
```

-

Sets: Unordered collections of unique elements.
```
my_set = {1, 2, 3}
```

-

Strings: Immutable sequences of characters.
```
my_string = "Hello"
```

-

8. Discuss How Python Interacts with the Operating System to Handle File Operations

Python interacts with the operating system to handle file operations through built-in functions and the `os` and `os.path`modules.

Reading from and Writing to Files:

Reading:
```
with open('example.txt', 'r') as file:
    content = file.read()
print(content)
```

-

Writing:
```
with open('example.txt', 'w') as file:
    file.write('Hello, World!')
```

-

Manipulating File Paths and Directories:

os Library:
```
import os

# Creating a directory
os.makedirs('my_directory', exist_ok=True)
```

```
# Listing files in a directory
files = os.listdir('my_directory')
print(files)

# Getting the current working directory
cwd = os.getcwd()
print(cwd)
```

-

Handling Exceptions:

Example:

```
try:
    with open('non_existent_file.txt', 'r') as file:
        content = file.read()
except FileNotFoundError as e:
    print(f"Error: {e}")
```

-

9. Compare and Contrast These Four Fundamental Python Data Structures

Lists:

- **Definition:** Ordered, mutable collections of elements.
- **Syntax:** `my_list = [1, 2, 3, 4]`
- **Use Cases:** Storing ordered collections of items, allowing duplicates.
- **Performance:** Fast index-based access, but slow for insertions and deletions in the middle.
- **Suitability:** Best for cases where order matters and you need dynamic sizing.

Tuples:

- **Definition:** Ordered, immutable collections of elements.
- **Syntax:** `my_tuple = (1, 2, 3, 4)`
- **Use Cases:** Storing fixed collections of items, ensuring data integrity.
- **Performance:** Faster than lists due to immutability; less memory usage.
- **Suitability:** Best for fixed collections of items where immutability is required.

Dictionaries:

- **Definition:** Unordered collections of key-value pairs.
- **Syntax:** `my_dict = {'a': 1, 'b': 2, 'c': 3}`

- **Use Cases:** Mapping unique keys to values, fast lookups.
- **Performance:** Fast average-time complexity for lookups, insertions, and deletions.
- **Suitability:** Best for associative arrays, where fast lookups by key are necessary.

Sets:

- **Definition:** Unordered collections of unique elements.
- **Syntax:** `my_set = {1, 2, 3, 4}`
- **Use Cases:** Removing duplicates from a collection, set operations (union, intersection).
- **Performance:** Fast membership testing, insertion, and deletion.
- **Suitability:** Best for membership testing and ensuring unique elements.

Unit - II

1. Define Universal Functions in the Context of NumPy

Universal Functions (ufuncs): Universal functions (ufuncs) in NumPy are functions that operate element-wise on ndarrays. They are highly optimized and support broadcasting, type casting, and various other features, making them a core feature of NumPy for performing fast vectorized operations.

Unary Universal Functions: These operate on a single input array.

`np.sqrt(x)`: Computes the square root of each element.

```
import numpy as np
x = np.array([1, 4, 9, 16])
print(np.sqrt(x))  # Output: [1. 2. 3. 4.]
```

-

`np.exp(x)`: Computes the exponential (e^x) of each element.

```
print(np.exp(x))  # Output: [2.71828183e+00 5.45981500e+01
8.10308393e+03 8.88611052e+06]
```

-

`np.abs(x)`: Computes the absolute value of each element.

```
x = np.array([-1, -2, 3, -4])
print(np.abs(x))  # Output: [1 2 3 4]
```

-

Binary Universal Functions: These operate on two input arrays.

`np.add(x, y)`: Adds corresponding elements of two arrays.

```
x = np.array([1, 2, 3, 4])
y = np.array([5, 6, 7, 8])
print(np.add(x, y))  # Output: [ 6  8 10 12]
```

-

`np.multiply(x, y)`: Multiplies corresponding elements of two arrays.

```
print(np.multiply(x, y))  # Output: [ 5 12 21 32]
```

-

`np.maximum(x, y)`: Compares two arrays and returns the maximum element-wise.

```
print(np.maximum(x, y))  # Output: [5 6 7 8]
```

-

2. Discuss the Various Methods Available in NumPy for Creating ndarrays

`np.array()`: Converts input data (list, tuple, etc.) into an ndarray.

```
data = [1, 2, 3, 4]
arr = np.array(data)
print(arr)  # Output: [1 2 3 4]
```

`np.zeros()`: Creates an array of the given shape and type, filled with zeros.

```
zeros_arr = np.zeros((2, 3))
print(zeros_arr)
# Output:
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

`np.ones()`: Creates an array of the given shape and type, filled with ones.

```
ones_arr = np.ones((2, 3))
print(ones_arr)
# Output:
# [[1. 1. 1.]
#  [1. 1. 1.]]
```

`np.arange()`: Creates an array with evenly spaced values within a given interval.

```
range_arr = np.arange(0, 10, 2)
print(range_arr)  # Output: [0 2 4 6 8]
```

`np.linspace()`: Creates an array with evenly spaced values over a specified range.

```
linspace_arr = np.linspace(0, 1, 5)
print(linspace_arr)  # Output: [0.   0.25 0.5  0.75 1.  ]
```

Examples:

Using `np.array()`:

```
data = [1, 2, 3, 4]
arr = np.array(data)
print(arr)  # Output: [1 2 3 4]
```

1.
    - **Purpose:** Converts a list into a NumPy array.

Using `np.zeros()`:

```
zeros_arr = np.zeros((2, 3))
print(zeros_arr)
# Output:
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

2.
    - **Purpose:** Creates a 2x3 array filled with zeros.

Using `np.linspace()`:

```
linspace_arr = np.linspace(0, 1, 5)
print(linspace_arr)  # Output: [0.   0.25 0.5  0.75 1.  ]
```

3.
    - **Purpose:** Generates 5 evenly spaced values between 0 and 1.

3. Explain How Indexing and Slicing Work in NumPy Arrays

Indexing: Accessing individual elements of the array using indices.

```
arr = np.array([1, 2, 3, 4, 5])
print(arr[0])  # Output: 1
print(arr[2])  # Output: 3
```

Slicing: Extracting subarrays from a given array using the colon (:) operator.

```
arr = np.array([1, 2, 3, 4, 5])
print(arr[1:4])  # Output: [2 3 4]
print(arr[:3])   # Output: [1 2 3]
print(arr[2:])   # Output: [3 4 5]
```

Multi-dimensional Arrays:

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr_2d[1, 2])  # Output: 6
print(arr_2d[0:2, 1:3])  # Output: [[2 3]
                         #          [5 6]]
```

4. Illustrate How NumPy Performs Element-wise Operations on Arrays

Addition:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)  # Output: [5 7 9]
```

Subtraction:

```
print(a - b)  # Output: [-3 -3 -3]
```

Multiplication:

```
print(a * b)  # Output: [ 4 10 18]
```

Division:

```
print(a / b)  # Output: [0.25 0.4  0.5 ]
```

Broadcasting: Broadcasting allows NumPy to perform operations on arrays of different shapes.

```
a = np.array([1, 2, 3])
b = np.array([[4], [5], [6]])
print(a + b)
# Output:
# [[ 5  6  7]
#  [ 6  7  8]
#  [ 7  8  9]]
```

In the example above, `a` (1D array) is broadcast across each row of `b` (2D array) to perform the addition.

5. Explain How Indexing and Slicing Work in NumPy Arrays

Indexing and slicing in NumPy arrays allow you to access and modify elements of the arrays efficiently.

Indexing:

Indexing allows you to access individual elements of an array. For a 1D array, you can simply use the index position.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
print(arr[0])  # Output: 1
print(arr[2])  # Output: 3
```

For multi-dimensional arrays, you can use a tuple of indices.

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr_2d[1, 2])  # Output: 6
print(arr_2d[2, 0])  # Output: 7
```

Slicing:

Slicing allows you to access a subset of an array. The syntax is `start:stop:step`.

```
arr = np.array([1, 2, 3, 4, 5])
print(arr[1:4])  # Output: [2 3 4]
print(arr[:3])   # Output: [1 2 3]
print(arr[2:])   # Output: [3 4 5]
print(arr[::2])  # Output: [1 3 5]
```

For multi-dimensional arrays, you can slice along each dimension.

```
arr_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(arr_2d[0:2, 1:3])  # Output: [[2 3]
                         #          [5 6]]
print(arr_2d[:, 1])      # Output: [2 5 8] (all rows, second column)
print(arr_2d[1, :])      # Output: [4 5 6] (second row, all columns)
```

6. Explain the Significance of Data Types in NumPy Arrays

Data types (dtypes) in NumPy arrays are crucial for performance and memory usage in numerical computations. By specifying data types, you can optimize memory consumption and computation speed.

Common Data Types:

- `np.int32`, `np.int64`: Signed integers.
- `np.float32`, `np.float64`: Floating-point numbers.
- `np.complex64`, `np.complex128`: Complex numbers.
- `np.bool_`: Boolean values.

Enhanced Performance:

Specifying data types can lead to significant performance improvements by reducing memory usage and increasing computation speed.

Example:

Creating arrays with different data types.

```
arr_int = np.array([1, 2, 3, 4], dtype=np.int32)
arr_float = np.array([1.0, 2.0, 3.0, 4.0], dtype=np.float32)
```

```
print(arr_int.dtype)   # Output: int32
print(arr_float.dtype) # Output: float32
```

Performing operations with specified data types.

```
arr1 = np.array([1, 2, 3], dtype=np.int32)
arr2 = np.array([4, 5, 6], dtype=np.int32)
result = arr1 + arr2
print(result)  # Output: [5 7 9]
print(result.dtype)  # Output: int32
```

Using appropriate data types ensures that the operations are optimized for performance.

7. Discuss the Various Methods Available in NumPy for Creating ndarrays

NumPy provides several methods to create ndarrays efficiently.

`np.array()`: Converts input data (list, tuple, etc.) into an ndarray.

```
data = [1, 2, 3, 4]
arr = np.array(data)
print(arr)  # Output: [1 2 3 4]
```

`np.zeros()`: Creates an array of the given shape and type, filled with zeros.

```
zeros_arr = np.zeros((2, 3))
print(zeros_arr)
# Output:
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

`np.ones()`: Creates an array of the given shape and type, filled with ones.

```
ones_arr = np.ones((2, 3))
print(ones_arr)
# Output:
# [[1. 1. 1.]
#  [1. 1. 1.]]
```

`np.arange()`: Creates an array with evenly spaced values within a given interval.

```
range_arr = np.arange(0, 10, 2)
print(range_arr)  # Output: [0 2 4 6 8]
```

`np.linspace()`: Creates an array with evenly spaced values over a specified range.

```
linspace_arr = np.linspace(0, 1, 5)
print(linspace_arr)  # Output: [0.   0.25 0.5  0.75 1.  ]
```

Examples:

Using `np.array()`:

```
data = [1, 2, 3, 4]
arr = np.array(data)
print(arr)  # Output: [1 2 3 4]
```

1.
    - **Purpose:** Converts a list into a NumPy array.

Using `np.zeros()`:

```
zeros_arr = np.zeros((2, 3))
print(zeros_arr)
# Output:
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

2.
    - **Purpose:** Creates a 2x3 array filled with zeros.

Using `np.linspace()`:

```
linspace_arr = np.linspace(0, 1, 5)
print(linspace_arr)  # Output: [0.   0.25 0.5  0.75 1.  ]
```

3.
    - **Purpose:** Generates 5 evenly spaced values between 0 and 1.

8. Describe How NumPy Handles Pseudorandom Number Generation

NumPy provides a submodule `numpy.random` that allows for the generation of pseudorandom numbers using various distributions.

Basic Functions:

`np.random.rand()`: Generates an array of random values between 0 and 1.

```
random_arr = np.random.rand(3, 2)
print(random_arr)
# Output: (random values)
# [[0.57019677 0.43860151]
#  [0.98837384 0.10204481]
#  [0.20887676 0.16130952]]
```

-

`np.random.randint()`: Generates random integers between a specified range.

```
random_int_arr = np.random.randint(0, 10, size=(3, 3))
print(random_int_arr)
# Output: (random values)
# [[6 9 1]
#  [4 1 3]
#  [4 7 6]]
```

-

`np.random.normal()`: Generates random samples from a normal (Gaussian) distribution.

```
normal_arr = np.random.normal(loc=0, scale=1, size=(2, 3))
print(normal_arr)
# Output: (random values)
# [[ 0.78236442 -0.91815536 -0.03009154]
#  [-0.04808602  0.37063443  0.21701132]]
```

-

Seeding the Random Number Generator: You can seed the random number generator to ensure reproducibility.

```
np.random.seed(42)
print(np.random.rand(2))
# Output: [0.37454012 0.95071431]
```

By using `numpy.random`, you can generate arrays of random numbers efficiently and reproducibly, which is essential for simulations, testing, and various other applications in data science and machine learning.

Unit - III

1. What are the Two Data Structures of Pandas? Explain with Examples.

Pandas primarily uses two data structures:

1. **Series:**
    - A Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.).
    - The labels in a Series are called the index.

Example:

```
import pandas as pd

data = [1, 2, 3, 4]
index = ['a', 'b', 'c', 'd']
series = pd.Series(data, index=index)
print(series)
# Output:
# a    1
# b    2
# c    3
# d    4
# dtype: int64
```

2. **DataFrame:**
    - A DataFrame is a two-dimensional labeled data structure with columns of potentially different types.
    - It is similar to a spreadsheet or SQL table.

Example:

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'San Francisco', 'Los Angeles']
}
```

```
df = pd.DataFrame(data)
print(df)
# Output:
#       Name  Age           City
# 0    Alice   25       New York
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles
```

2. List Five Indexing Options with DataFrames with Examples.

Indexing with a single column label:

```
print(df['Name'])
# Output:
# 0      Alice
# 1        Bob
# 2    Charlie
# Name: Name, dtype: object
```

Indexing with a list of column labels:

```
print(df[['Name', 'City']])
# Output:
#       Name           City
# 0    Alice       New York
# 1      Bob  San Francisco
# 2  Charlie    Los Angeles
```

Indexing using `.loc[]`:

- Used for label-based indexing.
  `print(df.loc[0])`

```
# Output:
# Name      Alice
# Age          25
# City    New York
# Name: 0, dtype: object

print(df.loc[:, ['Name', 'Age']])
```

```
# Output:
#       Name  Age
# 0    Alice   25
# 1      Bob   30
# 2  Charlie   35
```

Indexing using `.iloc[]`:

- Used for positional indexing.
  `print(df.iloc[0])`

```
# Output:
# Name      Alice
# Age          25
# City    New York
# Name: 0, dtype: object

print(df.iloc[:, [0, 2]])
# Output:
#       Name           City
# 0    Alice       New York
# 1      Bob  San Francisco
# 2  Charlie    Los Angeles
```

Boolean indexing:

```
print(df[df['Age'] > 25])
# Output:
#       Name  Age           City
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles
```

3. Explain Sorting and Ranking with Respect to Pandas.

Sorting:

Sorting refers to arranging data in a particular order, either ascending or descending.

Example:

```
# Sorting by a single column
sorted_df = df.sort_values(by='Age')
print(sorted_df)
```

```
# Output:
#       Name  Age           City
# 0    Alice   25       New York
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles

# Sorting by multiple columns
sorted_df = df.sort_values(by=['Age', 'Name'], ascending=[True,
False])
print(sorted_df)
# Output:
#       Name  Age           City
# 0    Alice   25       New York
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles
```

Ranking:

Ranking refers to assigning ranks to the data based on their values. The rank can be the average of the ranks or it can handle ties in various ways.

Example:

```
ranked_df = df['Age'].rank()
print(ranked_df)
# Output:
# 0    1.0
# 1    2.0
# 2    3.0
# Name: Age, dtype: float64

# Handling ties
ranked_df = df['Age'].rank(method='min')
print(ranked_df)
# Output:
# 0    1.0
# 1    2.0
# 2    3.0
# Name: Age, dtype: float64
```

4. Explain Dropping Entries from an Axis and Re-indexing in both DataFrame and Series.

Dropping Entries:

Dropping entries means removing specified labels from an axis.

Example:

```
# Dropping a column
dropped_df = df.drop('City', axis=1)
print(dropped_df)
# Output:
#       Name  Age
# 0    Alice   25
# 1      Bob   30
# 2  Charlie   35

# Dropping a row
dropped_df = df.drop(1, axis=0)
print(dropped_df)
# Output:
#       Name  Age         City
# 0    Alice   25     New York
# 2  Charlie   35  Los Angeles
```

Re-indexing:

Re-indexing is the process of conforming DataFrame to new index with optional filling logic.

Example:

```
# Re-indexing a DataFrame
new_index = [0, 1, 2, 3]
reindexed_df = df.reindex(new_index)
print(reindexed_df)
# Output:
#       Name   Age           City
# 0    Alice  25.0       New York
```

```
# 1      Bob  30.0  San Francisco
# 2  Charlie  35.0    Los Angeles
# 3      NaN   NaN            NaN

# Re-indexing a Series
new_index = ['a', 'b', 'c', 'd', 'e']
reindexed_series = series.reindex(new_index)
print(reindexed_series)
# Output:
# a    1.0
# b    2.0
# c    3.0
# d    4.0
# e    NaN
# dtype: float64
```

By re-indexing, you can change the structure of your data to align with the new indices, which is useful for aligning data from different sources or datasets.

5. List All Indexing Methods in DataFrames

Single column label:
python
Copy code

```
df['column_name']
```

1.

List of column labels:

```
df[['column1', 'column2']]
```

2.

.loc[] (label-based indexing):

```
df.loc[row_indexer, column_indexer]
```

3.

.iloc[] (integer-based indexing):

```
df.iloc[row_indexer, column_indexer]
```

4.

.at[] (access a single value for a row/column label pair):

```
df.at[row_label, column_label]
```

5.

.iat[] (access a single value for a row/column pair by integer position):

```
df.iat[row_position, column_position]
```

6.

Boolean indexing:

```
df[df['column_name'] > value]
```

7.

.ix[] (deprecated, but was used for a hybrid of .loc and .iloc):

```
df.ix[row_indexer, column_indexer]
```

8.

Callable indexing:

```
df[lambda x: x.index % 2 == 0]
```

9.

6. Brief about Fundamental Mechanics in Interacting with the Data in Series and DataFrame with Appropriate Examples

Series:

Creating a Series:

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
```

1.
2. **Accessing data:**

By index label:

```
print(s['a'])  # Output: 1
```

-

By integer location:

```
print(s[0])  # Output: 1
```

-

Slicing:

```
print(s['a':'c'])  # Output: a    1
                   #          b    2
                   #          c    3
                   # dtype: int64
```

3.

Boolean indexing:

```
print(s[s > 2])  # Output: c    3
                 #          d    4
                 #          e    5
                 # dtype: int64
```

4.

DataFrame:

Creating a DataFrame:
python
Copy code

```
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35],
'City': ['New York', 'San Francisco', 'Los Angeles']}
df = pd.DataFrame(data)
```

1.
2. **Accessing data:**

Single column:

```
print(df['Name'])
```

-

Multiple columns:

```
print(df[['Name', 'City']])
```

-

.loc[] (label-based):

```
print(df.loc[0])
# Output:
# Name      Alice
# Age          25
# City    New York
# Name: 0, dtype: object

print(df.loc[:, ['Name', 'Age']])
# Output:
#       Name  Age
# 0    Alice   25
# 1      Bob   30
# 2  Charlie   35
```

3.

.iloc[] (integer-based):

```
print(df.iloc[0])
# Output:
# Name      Alice
# Age          25
# City    New York
# Name: 0, dtype: object

print(df.iloc[:, [0, 2]])
# Output:
#       Name           City
```

```
# 0    Alice       New York
# 1      Bob  San Francisco
# 2  Charlie    Los Angeles
```

4.

Boolean indexing:

```
print(df[df['Age'] > 25])
# Output:
#       Name  Age           City
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles
```

5.

7. What are the Two Data Structures of Pandas? Explain with Examples

1. **Series:**
    - A one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.).

Example:

```
import pandas as pd

data = [1, 2, 3, 4]
index = ['a', 'b', 'c', 'd']
series = pd.Series(data, index=index)
print(series)
# Output:
# a    1
# b    2
# c    3
# d    4
# dtype: int64
```

2.
3. **DataFrame:**
    - A two-dimensional labeled data structure with columns of potentially different types. It is similar to a spreadsheet or SQL table.

Example:

```
data = {'Name': ['Alice', 'Bob', 'Charlie'], 'Age': [25, 30, 35],
'City': ['New York', 'San Francisco', 'Los Angeles']}
df = pd.DataFrame(data)
print(df)
# Output:
#       Name  Age           City
# 0    Alice   25       New York
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles
```

4.

8. Explain Sorting and Ranking with Respect to Pandas

Sorting:

Sorting refers to arranging data in a particular order, either ascending or descending.

Example:

```
# Sorting by a single column
sorted_df = df.sort_values(by='Age')
print(sorted_df)
# Output:
#       Name  Age           City
# 0    Alice   25       New York
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles

# Sorting by multiple columns
sorted_df = df.sort_values(by=['Age', 'Name'], ascending=[True,
False])
print(sorted_df)
# Output:
#       Name  Age           City
# 0    Alice   25       New York
# 1      Bob   30  San Francisco
# 2  Charlie   35    Los Angeles
```

Ranking:

Ranking refers to assigning ranks to the data based on their values. The rank can be the average of the ranks or it can handle ties in various ways.

Example:

```
ranked_df = df['Age'].rank()
print(ranked_df)
# Output:
# 0    1.0
# 1    2.0
# 2    3.0
# Name: Age, dtype: float64

# Handling ties
ranked_df = df['Age'].rank(method='min')
print(ranked_df)
# Output:
# 0    1.0
# 1    2.0
# 2    3.0
# Name: Age, dtype: float64
```

Unit - IV

1. How to Interact with Web APIs and Databases?

Interacting with Web APIs:

To interact with web APIs, you typically use the `requests` library in Python.

Example:

python

Copy code

```
import requests
```

```
# Make a GET request

response = requests.get('https://api.example.com/data')


# Check if the request was successful

if response.status_code == 200:

    data = response.json()  # Parse JSON data

    print(data)

else:

    print('Failed to retrieve data', response.status_code)
```

Interacting with Databases:

To interact with databases, you use libraries such as `sqlite3` for SQLite, `psycopg2` for PostgreSQL, or `pymysql` for MySQL.

Example with SQLite:

```
import sqlite3


# Connect to the database

conn = sqlite3.connect('example.db')

cursor = conn.cursor()


# Create a table

cursor.execute('''CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY
KEY, name TEXT, age INTEGER)''')
```

```
# Insert data

cursor.execute('''INSERT INTO users (name, age) VALUES (?, ?)''',
('Alice', 25))


# Query data

cursor.execute('SELECT * FROM users')

rows = cursor.fetchall()

for row in rows:

    print(row)


# Close the connection

conn.close()
```

2. Explain How to Read Microsoft Excel Files?

To read Microsoft Excel files, you can use the `pandas` library, which provides the `read_excel()` function.

Example:

```
import pandas as pd


# Read the entire Excel file

df = pd.read_excel('example.xlsx')


# Read a specific sheet
```

```
df_sheet = pd.read_excel('example.xlsx', sheet_name='Sheet1')


# Display the DataFrame

print(df)

print(df_sheet)
```

3. What are the Optional Arguments for Reading and Writing Data in Text Format?

Optional Arguments for Reading Data:

- `sep`: Delimiter to use.
- `header`: Row number(s) to use as the column names.
- `index_col`: Column(s) to set as index.
- `usecols`: Return a subset of the columns.
- `dtype`: Data type for data or columns.
- `engine`: Parser engine to use.
- `nrows`: Number of rows to read.
- `skiprows`: Number of rows to skip at the beginning.

Example:

```
df = pd.read_csv('example.csv', sep=',', header=0, index_col=0,
usecols=['A', 'B'], dtype={'A': int, 'B': float})
```

Optional Arguments for Writing Data:

- `sep`: String of length 1. Field delimiter for the output file.
- `na_rep`: Missing data representation.
- `columns`: Columns to write.
- `header`: Write out the column names.
- `index`: Write row names (index).
- `mode`: Python write mode.
- `encoding`: A string representing the encoding to use in the output file.

Example:

```
df.to_csv('example.csv', sep=',', na_rep='NaN', columns=['A', 'B'],
header=True, index=False, mode='w', encoding='utf-8')
```

4. List Text and Binary Data Loading Functions in Pandas with Descriptions

Text Data Loading Functions:

1. `read_csv()`: Read a comma-separated values (CSV) file into a DataFrame.
2. `read_table()`: Read general delimited file into a DataFrame.
3. `read_fwf()`: Read a table of fixed-width formatted lines into DataFrame.
4. `read_clipboard()`: Read text from the clipboard and pass to `read_table()`.

Binary Data Loading Functions:

1. `read_pickle()`: Load pickled pandas object (or any object) from a file.
2. `read_excel()`: Read an Excel file into a DataFrame.
3. `read_hdf()`: Read from the PyTables HDF5 file format.
4. `read_feather()`: Read a feather-format object.
5. `read_parquet()`: Load a parquet object.
6. `read_orc()`: Load an orc object.
7. `read_sas()`: Read from SAS files.
8. `read_spss()`: Read from SPSS files.

5. How Do You Read Text Files in Pieces and Write Data in Text Format?

Reading Text Files in Pieces:

You can read large files in chunks using the `chunksize` parameter in `read_csv()`.

Example:

```
chunk_size = 1000

chunks = pd.read_csv('large_file.csv', chunksize=chunk_size)


for chunk in chunks:
```

```
    process(chunk)  # Define your processing function
```

Writing Data in Text Format:

You can write DataFrame to a text file using `to_csv()`.

Example:

```
df.to_csv('output.txt', sep='\t')
```

6. Explain How to Write Data to Text Format

You can use `to_csv()`, `to_json()`, `to_html()`, etc., to write data to various text formats.

Example:

```
# Write to CSV

df.to_csv('output.csv', index=False)


# Write to JSON

df.to_json('output.json', orient='records', lines=True)


# Write to HTML

df.to_html('output.html')
```

7. Brief about Interaction with Database

Using `pandas`, you can interact with databases through SQLAlchemy.

Example:

```
from sqlalchemy import create_engine
```

```
import pandas as pd


# Create an engine

engine = create_engine('sqlite:///example.db')


# Write DataFrame to SQL database

df.to_sql('table_name', engine, if_exists='replace', index=False)


# Read SQL query into DataFrame

df_from_sql = pd.read_sql('SELECT * FROM table_name', engine)

print(df_from_sql)
```

8. How to Work with Various Delimited Formats

Use `read_csv()` with different `sep` parameters.

Example:

```
# Read CSV

df_csv = pd.read_csv('data.csv', sep=',')


# Read TSV

df_tsv = pd.read_csv('data.tsv', sep='\t')


# Read Pipe-separated

df_pipe = pd.read_csv('data.psv', sep='|')
```

9. What are the Optional Arguments for Reading and Writing Data in Text Format?

Refer to question 3 for details on optional arguments for reading and writing data in text format.

10. Explain How to Handle Microsoft Excel Files Using Pandas

Reading Excel Files: Use `read_excel()` to read Excel files.

Example:

```
import pandas as pd


# Read the entire Excel file

df = pd.read_excel('example.xlsx')


# Read a specific sheet

df_sheet = pd.read_excel('example.xlsx', sheet_name='Sheet1')


# Display the DataFrame

print(df)

print(df_sheet)
```

Writing to Excel Files: Use `to_excel()` to write DataFrame to Excel.

Example:

```
# Write to an Excel file

df.to_excel('output.xlsx', index=False)
```

```
# Write to a specific sheet

with pd.ExcelWriter('output.xlsx') as writer:

    df.to_excel(writer, sheet_name='Sheet1')

    df.to_excel(writer, sheet_name='Sheet2')
```

Unit - V

1. List Five String Methods in Pandas with Examples

Pandas provides various string manipulation functions for Series/Index. Here are five string methods:

`str.lower()`: Converts all characters to lowercase.

```
import pandas as pd
data = pd.Series(['Apple', 'Banana', 'Cherry'])
print(data.str.lower())
```

1.

`str.upper()`: Converts all characters to uppercase.
python
Copy code

```
print(data.str.upper())
```

2.

`str.contains()`: Checks if each string in the Series/Index contains a pattern/regex.
python
Copy code

```
print(data.str.contains('a'))
```

3.

`str.replace()`: Replaces occurrences of a pattern/regex with another string.
python
Copy code

```
print(data.str.replace('a', 'A'))
```

4.

`str.len()`: Computes the length of each string in the Series/Index.
python
Copy code
```
print(data.str.len())
```

5.

2. How to Handle Missing Data?

Handling missing data involves identifying and dealing with missing values in the dataset. Pandas provides several methods for this purpose:

Identifying Missing Data:
```
df.isnull()  # Returns a DataFrame of the same shape with Boolean values
df.isnull().sum()  # Summarizes the count of missing values in each column
```

1.

Dropping Missing Data:
python
Copy code
```
df.dropna()  # Drops any rows with missing values
df.dropna(axis=1)  # Drops any columns with missing values
df.dropna(thresh=2)  # Drops rows that have less than 2 non-null values
```

2.

Filling Missing Data:
python
Copy code
```
df.fillna(0)  # Fills missing values with 0
df.fillna(method='ffill')  # Forward fills the missing values
df.fillna(method='bfill')  # Backward fills the missing values
```

3.

3. How to Compute Indicator/Dummy Variables in Data Transforming?

Indicator or dummy variables are used to convert categorical data into a format that can be provided to ML algorithms to improve predictions.

Example:

```
import pandas as pd

df = pd.DataFrame({'color': ['red', 'blue', 'green']})
dummies = pd.get_dummies(df['color'])
print(dummies)
```

4. Discuss About Extension Data Types

Pandas extension data types extend the functionality of pandas by allowing the use of custom data types with Series and DataFrames. Some examples include `Categorical`, `Period`, `Interval`, `Sparse`, and `Nullable` integer types.

Example:

```
import pandas as pd

# Categorical
df = pd.DataFrame({'color': pd.Categorical(['red', 'blue', 'green'])})
print(df.dtypes)

# Nullable integer
df = pd.DataFrame({'values': pd.array([1, None, 3], dtype='Int64')})
print(df.dtypes)
```

5. Explain Data Transformation in Data Analysis

Data transformation is the process of converting data from its raw form into a format that is suitable for analysis. It involves cleaning, aggregating, and restructuring data. Some common transformations include normalization, scaling, encoding, and aggregating data.

Example:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [10, 20, 30, 40, 50]})
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
print(scaled_data)
```

7. How to Compute Indicator/Dummy Variables and Handle Outliers in Data Transforming?

Computing Dummy Variables: (See answer in section 3)

Handling Outliers: Outliers can be handled by removing, capping, or transforming the outliers to reduce their impact.

Example:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'data': [1, 2, 3, 4, 100]})
q1 = df['data'].quantile(0.25)
q3 = df['data'].quantile(0.75)
iqr = q3 - q1

# Removing outliers
df_filtered = df[~((df['data'] < (q1 - 1.5 * iqr)) | (df['data'] > (q3
+ 1.5 * iqr)))]
print(df_filtered)
```

8. Explain the Concepts with Respect to Data Transformation with Examples

a. Removing Duplicates

```
df = pd.DataFrame({'A': [1, 2, 2, 3], 'B': [4, 5, 5, 6]})
df_dedup = df.drop_duplicates()
print(df_dedup)
```

b. Replacing Values

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': ['a', 'b', 'c']})
df_replaced = df.replace({'A': {1: 100}, 'B': {'a': 'z'}})
print(df_replaced)
```

c. Renaming Axis Indexes

```
df = pd.DataFrame({'A': [1, 2, 3]}, index=['x', 'y', 'z'])
df_renamed = df.rename(index={'x': 'a', 'y': 'b'}, columns={'A':
'alpha'})
print(df_renamed)
```

d. Detecting and Filtering Outliers

(See example in section 7 under Handling Outliers)

e. Discretization and Binning

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5]})
df['binned'] = pd.cut(df['A'], bins=[0, 2, 4, 6], labels=['low',
'medium', 'high'])
print(df)
```

9. List Possible String Methods in Pandas with Descriptions

1. **`str.lower()`**: Converts strings in the Series/Index to lowercase.
2. **`str.upper()`**: Converts strings in the Series/Index to uppercase.
3. **`str.len()`**: Computes the length of each string in the Series/Index.
4. **`str.strip()`**: Removes leading and trailing whitespace.
5. **`str.split()`**: Splits strings around given separator/delimiter.

Example:

```
import pandas as pd

data = pd.Series([' Hello ', 'World ', ' pandas'])
print(data.str.strip())
print(data.str.upper())
print(data.str.len())
print(data.str.split())
```